```
def visualize_gp_belief(model, likelihood):
    with torch.no_grad():  ❶
        predictive_distribution = likelihood(model(xs))  ❶
        predictive_mean = predictive_distribution.mean  ❶
        predictive_upper, predictive_lower =  ❶
        <linearrow />    predictive_distribution.confidence_region()  ❶

    plt.figure(figsize=(8, 6))

    plt.plot(xs, ys, label="objective", c="r")
    plt.scatter(train_x, train_y, marker="x", c="k", label="observations")

    plt.plot(xs, predictive_mean, label="mean")  ❷
    plt.fill_between(
        xs.flatten(), predictive_upper, predictive_lower, alpha=0.3,
        label="95\% CI"
    )  ❸

    plt.legend(fontsize=15);
```

❶ Compute predictions.

❷ Plot the mean line.

❸ Plot the 95% CI region.

We have seen how this code works in Section 2.4.3 of Chapter 2, and now we are putting it into a convenient function. And with that, we are ready to implement our GP models and see how our choices affect the predictions that are being produced.

3.3.1 Using the zero mean function as the base strategy

The simplest form of the mean is a constant function at zero. In the absence of data, this function will produce zero as its default prediction. The zero mean function is used when there is no extra information about the objective function that we may incorporate into the GP as prior knowledge.

A GP with a zero mean function is implemented as follows:

```
class ConstantMeanGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ConstantMean()  ❶
        self.covar_module = gpytorch.kernels.RBFKernel()

    def forward(self, x):
        mean_x = self.mean_module(x)
        covar_x = self.covar_module(x)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)
```

❶ Constant mean function with the default value of zero.

Remember from Section 2.4.2 that in order to build a GP model with GPyTorch, we implement the `__init__()` and `forward()` methods. In the first method, we initialize our mean and covariance functions; in the second, we push the input `x` through these functions and return the

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>